YemenStay - Vollständige Projekt-Dokumentation

> ***Hotel-Buchungsplattform** für Unterkünfte im Jemen, gebaut mit Next.js, React, Supabase und Tailwind CSS.*

INHALTSVERZEICHNIS

1. Was ist dieses Projekt?
2. Technologien erklärt (für Einsteiger)
3. Projektstruktur - Jede Datei erklärt
4. Wie das Projekt startet und funktioniert
5. Die Architektur - Das große Bild
6. Provider-System (Globaler Zustand)
7. Seiten und ihre Funktionen
8. API-Routen (Backend)
9. Datenbank-Modell
10. Authentifizierung (Login-System)
11. Der komplette Buchungsablauf
12. Admin-Panel
13. Mehrsprachigkeit (i18n)
14. Styling und Design
15. Middleware und Routenschutz
16. Bild-Upload
17. E-Mail-System
18. Datenfluss-Diagramme
19. Wie Komponenten miteinander verbunden sind
20. Projekt starten

---

1. Was ist dieses Projekt?

YemenStay ist eine Webseite, auf der Benutzer:

- Hotels im Jemen durchsuchen können
- Zimmer ansehen und Preise vergleichen können
- Buchungen erstellen, bearbeiten und stornieren können
- Sich registrieren/anmelden können (oder als Gast buchen)
- Zwischen Englisch und Arabisch wechseln können

Für **Administratoren** gibt es ein Admin-Panel, in dem sie:

- Hotels erstellen, bearbeiten und löschen können
- Zimmer verwalten können
- Alle Buchungen einsehen und verwalten können
- Bilder hochladen können

---

2. Technologien erklärt

Was ist Next.js?

Next.js ist ein **Framework** (eine Art Werkzeugkasten) für React. Es erweitert React um:

- **Server-seitiges Rendering**: Seiten werden auf dem Server vorgerendert, bevor sie an den Browser geschickt werden (schneller, besser für SEO).
- **App Router**: Ein Dateisystem-basiertes Routing - jeder Ordner in `src/app/` wird automatisch zu einer URL.
- **API-Routen**: Backend-Logik direkt in der gleichen App (keine separate Backend-Anwendung nötig).

Was ist React?

React ist eine JavaScript-Bibliothek zum Bauen von Benutzeroberflächen. Kernkonzepte:

- **Komponenten**: Wiederverwendbare UI-Bausteine (z.B. ein Button, eine Navigation, eine Karte).
- **State (Zustand)**: Daten, die sich innerhalb einer Komponente ändern können (z.B. Eingabefeld-Wert).
- **Props**: Daten, die von einer Eltern-Komponente an eine Kind-Komponente übergeben werden.
- **Hooks**: Funktionen wie `useState`, `useEffect`, `useContext` - sie erlauben Komponenten, Zustand und Seiteneffekte zu haben.

Was ist Supabase?

Supabase ist eine **Backend-as-a-Service**-Plattform (wie Firebase, aber Open Source). Sie bietet:

- **PostgreSQL-Datenbank**: Hier werden Hotels, Zimmer und Buchungen gespeichert.
- **Authentifizierung**: Login/Registrierung mit E-Mail und Passwort.
- **Storage**: Dateispeicher für Bilder (Hotel-Fotos, Zimmer-Fotos).
- **JavaScript-Client**: Eine Bibliothek zum einfachen Zugriff auf alle Supabase-Dienste.

Was ist Tailwind CSS?

Tailwind CSS ist ein **Utility-First CSS-Framework**. Statt CSS-Klassen wie `.button-primary` selbst zu schreiben, nutzt man vorgefertigte Klassen direkt im HTML:

```
<!-- Statt eigene CSS-Klasse: -->
<button class="bg-blue-500 text-white px-4 py-2 rounded-lg hover:bg-blue-600">
  Klick mich
</button>
```

Jede Klasse macht **eine** Sache: `bg-blue-500` = blauer Hintergrund, `text-white` = weißer Text, usw.

Was ist Resend?

Resend ist ein E-Mail-Versanddienst. In diesem Projekt wird er genutzt, um **Buchungsbestätigungen** per E-Mail zu verschicken.

---

3. Projektstruktur

Hier ist **jede Datei** im Projekt mit einer Erklärung:

```
hotel-booking-nextjs/
│
├── package.json              ← Liste aller Abhängigkeiten + Befehle
├── package-lock.json         ← Exakte Versionen aller Abhängigkeiten
├── next.config.js            ← Next.js-Konfiguration (erlaubte Bild-Domains)
├── tailwind.config.js        ← Tailwind CSS-Konfiguration (Farben, Schriftarten)
├── postcss.config.js         ← PostCSS-Plugins (wird von Tailwind benötigt)
├── jsconfig.json             ← Pfad-Aliase (@/ = src/)
├── .env.local                ← Geheime Umgebungsvariablen (Supabase-Keys, etc.)
├── .gitignore                ← Dateien, die Git ignorieren soll
│
└── src/                      ← ALLER QUELLCODE lebt hier
    │
    ├── middleware.js          ← Prüft bei JEDEM Seitenaufruf die Authentifizierung
    │
    ├── app/                  ← Next.js App Router (Seiten + API)
    │   ├── layout.js         ← ROOT-LAYOUT: Umhüllt ALLE Seiten
    │   ├── page.js           ← STARTSEITE (/)
    │   ├── globals.css       ← Globale CSS-Stile
    │   ├── error.js          ← Fehlerseite (wenn etwas schiefgeht)
```

```
│   ├── loading.js        ← Ladeanimation
│   ├── not-found.js      ← 404-Seite
│   │
│   ├── HeroSearch.js     ← Suchleiste auf der Startseite
│   ├── DestinationCards.js ← Reiseziel-Karten
│   ├── FeaturedHotels.js ← Hotel-Karten auf der Startseite
│   ├── AboutSection.js   ← "Warum uns wählen"-Abschnitt
│   ├── StatsRow.js       ← Statistik-Anzeige (Anzahl Hotels, Städte etc.)
│   │
│   ├── hotel/
│   │   └── [id]/         ← Dynamische Route: /hotel/abc123
│   │       ├── page.js   ← Hotel-Detailseite
│   │       └── RoomList.js ← Zimmerliste-Komponente
│   │
│   ├── booking/
│   │   ├── page.js       ← Buchungsformular
│   │   ├── checkout-choice/
│   │   │   └── page.js   ← "Anmelden oder als Gast?"
│   │   └── confirmation/
│   │       └── page.js   ← Buchungsbestätigung
│   │
│   ├── login/
│   │   └── page.js       ← Login-Seite
│   │
│   ├── register/
│   │   └── page.js       ← Registrierungsseite
│   │
│   ├── my-bookings/
│   │   └── page.js       ← "Meine Buchungen" (geschützt)
│   │
│   ├── admin/            ← ADMIN-BEREICH (geschützt)
│   │   ├── layout.js     ← Admin-Layout (Sidebar + Inhalt)
│   │   ├── page.js       ← Admin-Dashboard
│   │   ├── AdminSidebar.js ← Seitenleiste
│   │   ├── admin.module.css ← Admin-spezifische Stile
│   │   ├── hotels/
│   │   │   └── page.js   ← Hotels verwalten
│   │   ├── rooms/
│   │   │   └── page.js   ← Zimmer verwalten
│   │   └── bookings/
│   │       └── page.js   ← Buchungen verwalten
│   │
│   └── api/              ← BACKEND-API-ROUTEN
│       ├── hotels/
│       │   ├── route.js       ← GET (alle Hotels) + POST (Hotel erstellen)
│       │   └── [id]/route.js  ← GET/PUT/DELETE einzelnes Hotel
│       ├── rooms/
│       │   ├── route.js       ← GET (alle Zimmer) + POST (Zimmer erstellen)
│       │   └── [id]/route.js  ← GET/PUT/DELETE einzelnes Zimmer
│       ├── bookings/
│       │   ├── route.js       ← GET (Buchungen) + POST (Buchung erstellen)
│       │   └── [id]/route.js  ← GET/PUT/DELETE einzelne Buchung
│       └── upload/
│           └── route.js       ← POST (Bild hochladen)
│
├── components/           ← WIEDERVERWENDBARE KOMPONENTEN
```

```
    │   ├── Navigation.js     ← Navigationsleiste (auf jeder Seite)
    │   ├── Footer.js         ← Fußzeile (auf jeder Seite)
    │   ├── LanguageSwitcher.js ← EN/AR Sprach-Umschalter
    │   └── providers/
    │       ├── AuthProvider.js     ← Authentifizierungs-Zustand (global)
    │       └── LanguageProvider.js ← Sprach-Zustand (global)
    │
    ├── lib/                  ← HILFSBIBLIOTHEKEN
    │   ├── email.js          ← E-Mail-Versand (Buchungsbestätigung)
    │   └── supabase/
    │       ├── client.js     ← Supabase-Client für den BROWSER
    │       ├── server.js     ← Supabase-Client für den SERVER
    │       └── middleware.js  ← Session-Verwaltung + Routenschutz
    │
    └── locales/              ← ÜBERSETZUNGSDATEIEN
        ├── en.json           ← Englische Texte
        └── ar.json           ← Arabische Texte
```

---

4. Wie das Projekt startet

Schritt 1: Der Benutzer ruft die URL auf

Wenn jemand `http://localhost:3000` öffnet, passiert Folgendes:

Schritt 2: Middleware wird ausgeführt

Datei: `src/middleware.js` → ruft `updateSession()` aus `src/lib/supabase/middleware.js` auf

```
Jede Anfrage → middleware.js → updateSession()
                                ├── Session erneuern (Cookies aktualisieren)
                                ├── Ist der Pfad geschützt? (/my-bookings, /admin)
                                │   ├── JA + nicht eingeloggt → Weiterleitung zu /login
                                │   └── JA + /admin + kein Admin → Weiterleitung zu /
                                └── Alles OK → Seite wird geladen
```

Die Middleware wird bei **jedem einzelnen Seitenaufruf** ausgeführt (außer für statische Dateien wie Bilder).

Schritt 3: Root-Layout wird geladen

Datei: `src/app/layout.js`

Das Root-Layout ist die **äußerste Hülle** der App. Es wird bei **jeder** Seite angezeigt:

```
<html>
  <body>
    <LanguageProvider>        ← Stellt Sprach-Funktionen bereit
      <AuthProvider>          ← Stellt Login-Status bereit
        <Navigation />        ← Navigationsleiste (oben)
        <main>{children}</main> ← HIER wird die aktuelle Seite eingefügt
        <Footer />            ← Fußzeile (unten)
      </AuthProvider>
    </LanguageProvider>
  </body>
</html>
```

Wichtig: `{children}` ist ein Platzhalter. Je nachdem welche URL aufgerufen wird, wird hier die entsprechende Seite eingefügt.

Schritt 4: Die jeweilige Seite wird gerendert

- URL `/` → `src/app/page.js` (Startseite)
- URL `/hotel/abc123` → `src/app/hotel/[id]/page.js`
- URL `/login` → `src/app/login/page.js`
- usw.

---

5. Die Architektur - Das große Bild

```
┌──────────────────────────────────────────────────────────────┐
│                        BROWSER (Frontend)                    │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐ │
│  │ Navigation   │  │ Aktuelle     │  │ Footer               │ │
│  │ (Header)     │  │ Seite        │  │                      │ │
│  │              │  │ (page.js)    │  │                      │ │
│  └──────┬───────┘  └──────┬───────┘  └──────────────────────┘ │
│         │                 │                                   │
│         ├─── useAuth() ───┤     ← Zugriff auf Login-Status    │
│         ├── useLanguage()─┤     ← Zugriff auf Sprache/Texte   │
│         │                 │                                   │
│         │          fetch('/api/...') ← API-Aufrufe            │
│         │                 │                                   │
└─────────────────────────┬─┴─────────────────────────────────┘
                          │ │
                          ▼ ▼
┌──────────────────────────────────────────────────────────────┐
│                   NEXT.JS SERVER (Backend)                    │
│                                                              │
│  ┌──────────┐  ┌──────────────────┐  ┌────────────────────┐ │
│  │Middleware │  │ API-Routen       │  │ Server Components  │ │
│  │(Schutz)  │  │ /api/hotels      │  │ (page.js ohne      │ │
│  │          │  │ /api/rooms       │  │  'use client')     │ │
│  │          │  │ /api/bookings    │  │                    │ │
│  │          │  │ /api/upload      │  │                    │ │
│  └──────────┘  └──────────┬───────┘  └──────────┬─────────┘ │
│                           │                     │           │
└───────────────────────────┼─────────────────────┼───────────┘
                            │                     │
                            ▼                     ▼
┌──────────────────────────────────────────────────────────────┐
│                       SUPABASE (Cloud)                       │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐ │
│  │ PostgreSQL   │  │ Auth         │  │ Storage              │ │
│  │ Datenbank    │  │ (Login)      │  │ (Bilder)             │ │
│  │              │  │              │  │                      │ │
│  │ - hotels     │  │ - Benutzer   │  │ - hotel-images/      │ │
│  │ - rooms      │  │ - Sessions   │  │ - room-images/       │ │
│  │ - bookings   │  │ - Tokens     │  │                      │ │
│  └──────────────┘  └──────────────┘  └──────────────────────┘ │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Server Components vs. Client Components

In Next.js gibt es zwei Arten von Komponenten:

Server Components (Standard):

- Werden auf dem **Server** gerendert
- Können **direkt** auf die Datenbank zugreifen
- Können **keine** React-Hooks (useState, useEffect) verwenden
- Können **keine** Browser-APIs nutzen (localStorage, window, etc.)
- Beispiel: `src/app/page.js` (Startseite), `src/app/hotel/[id]/page.js`

Client Components (mit `'use client'` am Anfang):

- Werden im **Browser** gerendert
- Können React-Hooks verwenden
- Können mit dem Benutzer interagieren (Klicks, Eingaben, etc.)
- Müssen über `fetch()` auf die API zugreifen (nicht direkt auf die Datenbank)
- Beispiel: `Navigation.js`, `BookingPage`, `LoginPage`

---

6. Provider-System (Globaler Zustand)

Was sind Provider?

Provider sind Komponenten, die **Daten an alle ihre Kindkomponenten** weitergeben, egal wie tief diese verschachtelt sind. Ohne Provider müsste man Daten Schicht für Schicht nach unten durchreichen ("Prop Drilling").

AuthProvider (`src/components/providers/AuthProvider.js`)

Zweck: Macht den Login-Status überall in der App verfügbar.

Wie es funktioniert:

```
1. AuthProvider wird in layout.js eingebunden
   ↓
2. Beim Laden: Fragt Supabase "Wer ist eingeloggt?"
   → supabase.auth.getUser()
   ↓
3. Speichert den Benutzer in useState:
   → user = { id: "...", email: "..." } oder null
   ↓
4. Hört auf Login/Logout-Ereignisse:
   → supabase.auth.onAuthStateChange()
   → Aktualisiert user automatisch
   ↓
5. Stellt bereit:
   → user:     Der aktuelle Benutzer (oder null)
   → loading:  Wird noch geladen? (true/false)
   → signOut:  Funktion zum Ausloggen
```

Benutzung in jeder Komponente:

```
import { useAuth } from '@/components/providers/AuthProvider'

function MeineKomponente() {
  const { user, loading, signOut } = useAuth()

  if (loading) return <p>Laden...</p>
```

```
  if (!user) return <p>Nicht eingeloggt</p>
  return <p>Hallo {user.email}!</p>
}
```

LanguageProvider (`src/components/providers/LanguageProvider.js`)

Zweck: Macht die aktuelle Sprache und die Übersetzungsfunktion überall verfügbar.

Wie es funktioniert:

```
1. LanguageProvider wird in layout.js eingebunden
   ↓
2. Beim Laden: Liest gespeicherte Sprache aus localStorage
   → 'en' oder 'ar' (Standard: 'en')
   ↓
3. Lädt die Übersetzungsdateien:
   → en.json und ar.json werden importiert
   ↓
4. Die t()-Funktion übersetzt Schlüssel:
   → t('nav.destinations') → "Destinations" (EN) oder "الوجهات" (AR)
   ↓
5. Bei Sprachwechsel:
   → Sprache in localStorage speichern
   → HTML-Attribut lang="" aktualisieren
   → HTML-Attribut dir="" aktualisieren (ltr/rtl)
   ↓
6. Stellt bereit:
   → language:    'en' oder 'ar'
   → setLanguage: Funktion zum Sprachwechsel
   → t:           Übersetzungsfunktion
   → isRTL:       true wenn Arabisch (Rechts-nach-Links)
```

Wie die t()-Funktion intern funktioniert:

```
// en.json enthält:
{
  "nav": {
    "destinations": "Destinations",
    "hotels": "Hotels"
  }
}

// t('nav.destinations') macht Folgendes:
// 1. Teilt den Schlüssel auf: ['nav', 'destinations']
// 2. Navigiert durch das JSON-Objekt:
//    translations['en'] → { nav: { destinations: "Destinations" } }
//    → .nav → { destinations: "Destinations" }
//    → .destinations → "Destinations"
// 3. Gibt "Destinations" zurück
```

Wie Provider zusammenhängen (layout.js):

```
// src/app/layout.js
<LanguageProvider>        // ÄUSSERER Provider
  <AuthProvider>          // INNERER Provider
```

```
    <Navigation />        // ← Hat Zugriff auf BEIDE Provider
    <main>{children}</main> // ← Jede Seite hat Zugriff auf BEIDE
    <Footer />            // ← Hat Zugriff auf BEIDE Provider
  </AuthProvider>
</LanguageProvider>
```

Regel: Jede Komponente, die INNERHALB eines Providers liegt, kann dessen Daten nutzen.

---

7. Seiten und ihre Funktionen

7.1 Startseite (`src/app/page.js`)

URL: `/` **Typ:** Server Component (kein `'use client'`)

Was passiert beim Laden:

1. Server erstellt einen Supabase-Client
2. Holt alle Hotels aus der Datenbank: `supabase.from('hotels').select('*, rooms(price)')`
3. Berechnet den **Mindestpreis** pro Hotel (billigstes Zimmer)
4. Zählt Hotels pro Stadt für die Reiseziel-Karten
5. Gibt die vorbereiteten Daten an die Kind-Komponenten weiter

Aufbau der Seite:

```
┌──────────────────────────────────────┐
│ HERO SECTION (großes Bild + Suchleiste)  │
│  ├── HeroSearch (Suchformular)           │
│  └── StatsRow (Anzahl Hotels/Städte)     │
├──────────────────────────────────────┤
│ REISEZIELE (Destination Cards)           │
│  └── DestinationCards (Städte-Karten)    │
├──────────────────────────────────────┤
│ HOTELS (Featured Hotels)                 │
│  └── FeaturedHotels (Hotel-Karten)       │
├──────────────────────────────────────┤
│ ÜBER UNS (Why Choose Us)                 │
│  └── AboutSection (Feature-Icons)        │
└──────────────────────────────────────┘
```

7.2 Hotel-Detailseite (`src/app/hotel/[id]/page.js`)

URL: `/hotel/[id]` (z.B. `/hotel/abc-123-def`) **Typ:** Server Component

Was `[id]` bedeutet: Die eckigen Klammern machen dies zu einer **dynamischen Route**. `[id]` wird durch die tatsächliche Hotel-ID ersetzt. Next.js gibt diese ID als `params.id` an die Seite weiter.

Was passiert:

1. Hotel-Daten werden mit `params.id` aus Supabase geladen
2. Wenn Hotel nicht existiert → 404-Seite (`notFound()`)
3. Zimmer für dieses Hotel werden geladen
4. **RoomList** (Client Component) zeigt die Zimmer an

RoomList-Komponente (`src/app/hotel/[id]/RoomList.js`):

- Zeigt jedes Zimmer als Karte mit Bild, Kapazität und Preis
- "Book Now"-Button → Speichert Hotel + Zimmer in `sessionStorage`
- Prüft ob der Benutzer eingeloggt ist:

- **JA** → Weiterleitung zu `/booking`
- **NEIN** → Weiterleitung zu `/booking/checkout-choice`

7.3 Checkout-Choice (`src/app/booking/checkout-choice/page.js`)

URL: `/booking/checkout-choice` **Zweck:** Gibt nicht eingeloggten Benutzern die Wahl:

```
┌──────────────────────────────────────┐
│  Wie möchten Sie fortfahren?         │
│                                      |
│  [Anmelden]  [Registrieren]  [Als Gast] │
└──────────────────────────────────────┘
```

- "Anmelden" → `/login?redirect=/booking`
- "Registrieren" → `/register?redirect=/booking`
- "Als Gast" → `/booking` (ohne Login)

7.4 Buchungsformular (`src/app/booking/page.js`)

URL: `/booking` **Typ:** Client Component

Was passiert:

1. Liest Hotel + Zimmer aus `sessionStorage` (wurden von RoomList gespeichert)
2. Wenn nichts in sessionStorage → Weiterleitung zur Startseite
3. Wenn Benutzer eingeloggt → E-Mail wird automatisch ausgefüllt
4. Benutzer füllt das Formular aus (Vorname, Nachname, Check-in, Check-out)
5. Bei Absenden: `POST /api/bookings` wird aufgerufen
6. Server prüft **Verfügbarkeit** (kein Doppelbuchung)
7. Bei Erfolg: Buchungsdaten in sessionStorage → Weiterleitung zu `/booking/confirmation`

7.5 Buchungsbestätigung (`src/app/booking/confirmation/page.js`)

URL: `/booking/confirmation` **Zweck:** Zeigt die Bestätigung nach erfolgreicher Buchung an.

- Liest Buchungsdaten aus sessionStorage
- Zeigt Hotel, Zimmer, Preis, Daten an
- Löscht die temporären Daten aus sessionStorage
- Link zurück zur Startseite

7.6 Login-Seite (`src/app/login/page.js`)

URL: `/login` oder `/login?redirect=/booking` **Zweck:** Benutzer mit E-Mail + Passwort anmelden.

Ablauf:

1. Benutzer gibt E-Mail und Passwort ein
2. `supabase.auth.signInWithPassword()` wird aufgerufen
3. Bei Erfolg: Weiterleitung zur `redirect` -URL (oder `/`)
4. `router.refresh()` aktualisiert die Seite (damit AuthProvider den neuen Status bemerkt)

7.7 Registrierungsseite (`src/app/register/page.js`)

URL: `/register` **Zweck:** Neuen Benutzer erstellen.

Ablauf:

1. Benutzer gibt E-Mail + Passwort + Passwort-Bestätigung ein
2. Validierung: Passwörter müssen übereinstimmen, mind. 6 Zeichen
3. `supabase.auth.signUp()` wird aufgerufen
4. Supabase sendet eine Bestätigungs-E-Mail

5. Benutzer sieht "Registrierung erfolgreich! Bitte E-Mail bestätigen."

7.8 Meine Buchungen (`src/app/my-bookings/page.js`)

URL: `/my-bookings` (geschützt - nur für eingeloggte Benutzer) **Zweck:** Zeigt alle Buchungen des aktuellen Benutzers.

Funktionen:

- **Anzeigen:** Hotel, Zimmer, Preis, Daten, Nächte, Gesamtpreis
- **Bearbeiten:** Check-in/Check-out Daten ändern (`PUT /api/bookings/[id]`)
- **Stornieren:** Buchung löschen (`DELETE /api/bookings/[id]`) mit Bestätigungsdialog

---

8. API-Routen (Backend)

API-Routen sind Dateien in `src/app/api/` , die als **Backend-Endpunkte** fungieren. Sie empfangen HTTP-Anfragen und geben JSON-Antworten zurück.

8.1 Hotels-API

Datei: `src/app/api/hotels/route.js`

Methode	URL	Was passiert	Auth nötig?
GET	`/api/hotels`	Alle Hotels abrufen (optional nach Stadt filtern)	Nein
POST	`/api/hotels`	Neues Hotel erstellen	Ja

Datei: `src/app/api/hotels/[id]/route.js`

Methode	URL	Was passiert	Auth nötig?
GET	`/api/hotels/abc123`	Ein Hotel abrufen	Nein
PUT	`/api/hotels/abc123`	Hotel aktualisieren	Ja
DELETE	`/api/hotels/abc123`	Hotel löschen	Ja

Beispiel - GET alle Hotels:

```
// Der Server...
export async function GET(request) {
  const supabase = await createClient()              // 1. Datenbank-Verbindung
  const { searchParams } = new URL(request.url)       // 2. URL-Parameter lesen
  const city = searchParams.get('city')                // 3. Stadt-Filter?

  let query = supabase.from('hotels').select('*')      // 4. Abfrage aufbauen

  if (city && city !== 'Alle') {
    query = query.eq('city', city)                     // 5. Nach Stadt filtern
  }

  const { data, error } = await query.order('name')   // 6. Abfrage ausführen

  if (error) {
    return NextResponse.json({ error: error.message }, { status: 500 })  // Fehler
  }
```

```
  return NextResponse.json(data)                        // 7. Hotels als JSON zurückgeben
}
```

8.2 Zimmer-API

Datei: `src/app/api/rooms/route.js` + `[id]/route.js`

Gleiche Struktur wie Hotels. Zusätzlich:

- GET kann nach `hotel_id` filtern
- Bei GET wird auch das zugehörige Hotel mitgeladen (JOIN):

```
supabase.from('rooms').select(`*, hotels (id, name, city)`)
```

8.3 Buchungs-API

Datei: `src/app/api/bookings/route.js`

POST - Buchung erstellen (der wichtigste Endpunkt):

```
Anfrage kommt rein
    ↓
1. Benutzer prüfen (optional - Gäste dürfen auch buchen)
    ↓
2. Pflichtfelder prüfen: room_id, firstname, lastname, email, check_in, check_out
    ↓
3. Daten validieren: check_out muss NACH check_in sein
    ↓
4. VERFÜGBARKEITSPRÜFUNG:
   Suche Buchungen, die sich mit dem gewünschten Zeitraum überschneiden:
   → WHERE room_id = gewünschtes_zimmer
   → AND check_in < gewünschtes_check_out     (beginnt VOR unserem Ende)
   → AND check_out > gewünschtes_check_in     (endet NACH unserem Start)
   → Wenn Ergebnisse gefunden → Zimmer ist BELEGT → 409 Fehler
    ↓
5. Buchung in Datenbank speichern
    ↓
6. Bestätigungs-E-Mail senden (im Hintergrund)
    ↓
7. Buchungsdaten als JSON zurückgeben (Status 201)
```

Verfügbarkeitsprüfung visualisiert:

```
Zeitstrahl:  Jan 1 ── Jan 5 ── Jan 10 ── Jan 15 ── Jan 20

Bestehende Buchung:     |└└└
```

8.4 Upload-API

Datei: `src/app/api/upload/route.js`

Zweck: Bilder hochladen für Hotels/Zimmer.

Ablauf:

1. Benutzer muss eingeloggt sein
2. Formular-Daten empfangen (Datei + Bucket-Name)
3. Validierung: Nur JPEG/PNG/WebP/GIF, max 5MB
4. Eindeutigen Dateinamen generieren (Zeitstempel + Zufallsstring)
5. In Supabase Storage hochladen
6. Öffentliche URL zurückgeben

---

9. Datenbank-Modell

Tabellen und ihre Beziehungen

```
┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│    auth.users    │     │      hotels      │     │      rooms       │
│ (von Supabase Auth) │     │                  │     │                  │
├──────────────────┤     ├──────────────────┤     ├──────────────────┤
│ id (UUID) PK     │     │ id (UUID) PK     │     │ id (UUID) PK     │
│ email            │     │ name             │◄────│ hotel_id (FK)    │
│ encrypted_password │     │ city             │     │ name             │
│ created_at       │     │ description      │     │ price            │
│ ...              │     │ image_url        │     │ capacity         │
│                  │     │ created_at       │     │ image_url        │
│                  │     │ updated_at       │     │ created_at       │
└──────────────────┘     └──────────────────┘     │ updated_at       │
        │                                          └──────────────────┘
        │                                                   │
        │                                                   │
        │            ┌──────────────────┐                   │
        │            │     bookings     │                   │
        │            ├──────────────────┤                   │
        └────────────│ user_id (FK, null) │                   │
                     │ room_id (FK)     │───────────────────┘
                     │ id (UUID) PK     │
                     │ firstname        │
                     │ lastname         │
                     │ email            │
                     │ check_in (date)  │
                     │ check_out (date) │
                     │ created_at       │
                     │ updated_at       │
                     └──────────────────┘
```

Beziehungen erklärt:

- Ein **Hotel** hat viele **Zimmer** (1:N) → `rooms.hotel_id` verweist auf `hotels.id`
- Ein **Zimmer** hat viele **Buchungen** (1:N) → `bookings.room_id` verweist auf `rooms.id`
- Ein **Benutzer** hat viele **Buchungen** (1:N) → `bookings.user_id` verweist auf `auth.users.id`
- `user_id` kann `null` sein (für Gast-Buchungen ohne Login)

Wie JOINs funktionieren (Supabase-Syntax):

```
// Beim Laden einer Buchung werden Zimmer UND Hotel mitgeladen:
supabase.from('bookings').select(`
  *,                    ← alle Buchungsfelder
  rooms (               ← JOIN mit rooms-Tabelle
    id,
    name,
    price,
    hotels (            ← JOIN mit hotels-Tabelle (über rooms)
      id,
      name,
      city
    )
  )
`)
// Ergebnis:
// {
//   id: "buchung-123",
//   firstname: "Max",
//   rooms: {
//     name: "Deluxe Suite",
//     price: 120,
//     hotels: {
//       name: "Grand Hotel Sanaa",
//       city: "Sana'a"
//     }
//   }
// }
```

---

10. Authentifizierung (Login-System)

Gesamtübersicht

```
┌──────────────────────────────────────────────────────────┐
│                    AUTHENTIFIZIERUNGSFLUSS                 │
│                                                            │
│  Registrierung:                                            │
│  ┌──────────┐    supabase.auth.signUp()   ┌──────────┐    │
│  │ Register │ ──────────────────────────→ │ Supabase │    │
│  │ Seite    │                              │ Auth     │    │
│  └──────────┘                              └────┬─────┘    │
│                                                 │          │
│                              Bestätigungs-Email ↓          │
│                                            ┌──────────┐    │
│                                            │ E-Mail-  │    │
│                                            │ Postfach │    │
│                                            └──────────┘    │
│                                                            │
│  Login:                                                    │
│  ┌──────────┐  signInWithPassword()  ┌──────────┐         │
│  │ Login    │ ──────────────────────→ │ Supabase │         │
│  │ Seite    │ ←────────────────────── │ Auth     │         │
│  └────┬─────┘    Session + Cookies    └──────────┘         │
│       │                                                    │
│       ↓                                                    │
```

```
│   ┌──────────────────────────────────────────┐        │
│   │ AuthProvider bemerkt Login:                  │        │
│   │ onAuthStateChange() wird ausgelöst           │        │
│   │ → user-Zustand wird aktualisiert             │        │
│   │ → Navigation zeigt "Logout" statt "Sign In"  │        │
│   │ → Geschützte Seiten werden zugänglich        │        │
│   └──────────────────────────────────────────┘        │
│                                                          │
│   Geschützte Route aufrufen:                             │
│   ┌────────┐    ┌──────────┐    ┌─────────┐            │
│   │Browser │ →  │ Middleware  │ → │ Supabase  │            │
│   │        │    │ Prüft      │    │ getUser() │            │
│   │        │    │ Session    │    └────┬────┘            │
│   │        │    └─────┬────┘         │                  │
│   │        │          │          Hat Session?              │
│   │        │          │          JA → Seite laden          │
│   │        │          │          NEIN → /login             │
│   └────────┘          │                                   │
│                       │          Ist Admin-Route?          │
│                       │          JA → E-Mail in ADMIN_EMAILS? │
│                       │               JA → Zugriff         │
│                       │               NEIN → Startseite    │
└──────────────────────────────────────────────────────────┘
```

Zwei verschiedene Supabase-Clients

Warum zwei? Browser und Server haben unterschiedliche Möglichkeiten für Cookie-Zugriff.

Browser-Client (`src/lib/supabase/client.js`):

```
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL,     // Supabase-URL
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY // Öffentlicher API-Schlüssel
  )
}
```

- Wird in **Client Components** verwendet (Login, Register, etc.)
- Verwaltet Cookies automatisch über den Browser

Server-Client (`src/lib/supabase/server.js`):

```
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export async function createClient() {
  const cookieStore = await cookies()      // Next.js Cookie-API
  return createServerClient(URL, KEY, {
    cookies: {
      getAll() { return cookieStore.getAll() },     // Cookies lesen
      setAll(cookiesToSet) { /* Cookies setzen */ }  // Cookies schreiben
    }
  })
}
```

- Wird in **Server Components** und **API-Routen** verwendet
- Muss Cookies manuell über Next.js `cookies()` API verwalten

---

11. Der komplette Buchungsablauf

Hier ist der **komplette Weg** von "Hotel ansehen" bis "Buchung bestätigt":

```
SCHRITT 1: HOTEL AUSWÄHLEN
═══════════════════════════
Benutzer auf Startseite (page.js)
    │
    ├── Server lädt Hotels aus Supabase
    ├── FeaturedHotels zeigt Hotel-Karten an
    │
    └── Benutzer klickt auf ein Hotel
            ↓
        /hotel/abc-123 wird geladen (hotel/[id]/page.js)
            │
            ├── Server lädt Hotel-Details + Zimmer
            ├── RoomList zeigt verfügbare Zimmer
            │
            └── Benutzer klickt "Book Now" auf einem Zimmer
                    ↓

SCHRITT 2: ZWISCHENSPEICHERUNG
══════════════════════════════
RoomList.handleSelectRoom(room):
    │
    ├── sessionStorage.setItem('selectedHotel', JSON.stringify(hotel))
    ├── sessionStorage.setItem('selectedRoom', JSON.stringify(room))
    │
    └── Ist der Benutzer eingeloggt?
        ├── JA → router.push('/booking')
        └── NEIN → router.push('/booking/checkout-choice')
                        ↓

SCHRITT 3 (nur für Gäste): ENTSCHEIDUNG
═══════════════════════════════════════
/booking/checkout-choice:
    │
    ├── [Anmelden] → /login?redirect=/booking
    ├── [Registrieren] → /register?redirect=/booking
    └── [Als Gast] → /booking
            ↓

SCHRITT 4: BUCHUNGSFORMULAR
═══════════════════════════
/booking (booking/page.js):
    │
    ├── Liest Hotel + Zimmer aus sessionStorage
    ├── Wenn eingeloggt: E-Mail wird vorausgefüllt
    │
    ├── Benutzer füllt aus:
    │   ├── Vorname
    │   ├── Nachname
    │   ├── E-Mail (bei Login automatisch)
```

```
    │   ├── Check-in Datum
    │   └── Check-out Datum
    │
    └── Klickt "Jetzt buchen"
            ↓

SCHRITT 5: API-AUFRUF
═══════════════════
fetch('/api/bookings', {
    method: 'POST',
    body: { room_id, firstname, lastname, email, check_in, check_out }
})
    ↓
Server (api/bookings/route.js):
    │
    ├── Pflichtfelder prüfen
    ├── Daten validieren (check_out > check_in)
    │
    ├── VERFÜGBARKEITSPRÜFUNG:
    │   → Überschneidende Buchungen suchen
    │   → Gefunden? → 409 "Zimmer nicht verfügbar"
    │   → Nicht gefunden? → Weiter
    │
    ├── Buchung in Datenbank speichern
    │   → user_id = eingeloggter User oder null (Gast)
    │
    ├── Bestätigungs-E-Mail senden (im Hintergrund, nicht blockierend)
    │
    └── Buchungsdaten zurückgeben (Status 201)
            ↓

SCHRITT 6: BESTÄTIGUNG
═══════════════════
Zurück im Browser (booking/page.js):
    │
    ├── Buchungsdaten in sessionStorage speichern
    ├── Hotel + Zimmer aus sessionStorage löschen
    │
    └── Weiterleitung zu /booking/confirmation
            ↓
/booking/confirmation (confirmation/page.js):
    │
    ├── Liest Buchungsdaten aus sessionStorage
    ├── Löscht sie danach (einmalige Anzeige)
    │
    └── Zeigt an:
        ├── ✓ Buchung erfolgreich!
        ├── Hotel, Zimmer, Preis, Daten
        └── [Zurück zur Startseite]
```

---

12. Admin-Panel

Zugang

Das Admin-Panel ist unter `/admin` erreichbar. Zugang haben nur Benutzer, deren E-Mail in der Umgebungsvariable `ADMIN_EMAILS` steht.

Prüfung in der Middleware (`src/lib/supabase/middleware.js`):

```
if (request.nextUrl.pathname.startsWith('/admin') && user) {
  const adminEmails = (process.env.ADMIN_EMAILS || '').split(',').map(e => e.trim())
  if (!adminEmails.includes(user.email)) {
    // Nicht-Admin → Weiterleitung zur Startseite
    return NextResponse.redirect(url)
  }
}
```

Layout

Datei: `src/app/admin/layout.js`

Das Admin-Layout ersetzt das normale Navigation/Footer-Layout:

```
┌──────────────────────────────────────────┐
│  ┌──────────────┐  ┌──────────────────────┐  │
│  │  Sidebar     │  │  Inhalt              │  │
│  │              │  │                       │  │
│  │  Dashboard   │  │  (Wechselt je nach    │  │
│  │  Hotels      │  │   URL)               │  │
│  │  Zimmer      │  │                       │  │
│  │  Buchungen   │  │                       │  │
│  │              │  │                       │  │
│  │  ← Zurück    │  │                       │  │
│  └──────────────┘  └──────────────────────┘  │
└──────────────────────────────────────────┘
```

Warum separates Layout? Navigation und Footer der öffentlichen Seite werden im Admin-Bereich ausgeblendet (sie prüfen `pathname.startsWith('/admin')`). Stattdessen gibt es eine Admin-Sidebar.

Dashboard (`/admin`)

Zeigt Übersicht:

- Anzahl Hotels, Zimmer, Buchungen
- Letzte 5 Buchungen in einer Tabelle

Daten werden parallel geladen:

```
const [hotelsRes, roomsRes, bookingsRes] = await Promise.all([
  fetch('/api/hotels'),
  fetch('/api/rooms'),
  fetch('/api/bookings?all=true'),  // ?all=true → ALLE Buchungen, nicht nur eigene
])
```

Hotel-/Zimmer-/Buchungsverwaltung

Alle drei Verwaltungsseiten folgen dem gleichen Muster:

```
1. Seite lädt → fetch('/api/...') → Daten in useState speichern
2. Tabelle zeigt alle Einträge an
3. [Neu erstellen] Button → Modal-Dialog öffnet sich
4. Modal hat Formularfelder → Absenden → POST /api/...
5. [Bearbeiten] Button in Tabellenzeile → Modal mit vorausgefüllten Daten
6. Absenden → PUT /api/.../[id]
```

```
7. [Löschen] Button → Bestätigungsdialog → DELETE /api/.../[id]
8. Nach jeder Aktion: Daten neu laden
```

---

13. Mehrsprachigkeit (i18n)

Unterstützte Sprachen

- **Englisch (en)** - Standard, Links-nach-Rechts (LTR)
- **Arabisch (ar)** - Rechts-nach-Links (RTL)

Übersetzungsdateien

Datei: `src/locales/en.json` und `src/locales/ar.json`

Struktur (Auszug):

```json
{
  "nav": {
    "home": "Home",
    "destinations": "Destinations",
    "hotels": "Hotels",
    "about": "About",
    "signIn": "Sign In",
    "logout": "Logout",
    "myBookings": "My Bookings",
    "bookNow": "Book Now"
  },
  "home": {
    "heroTitle": "Experience Yemen's Timeless Beauty",
    "heroSubtitle": "Book unique stays...",
    "searchPlaceholder": "Select a city..."
  },
  "hotel": {
    "bookNow": "Book Now",
    "perNight": "night",
    "persons": "persons",
    "noRooms": "No rooms available"
  },
  "admin": {
    "dashboard": "Dashboard",
    "hotels": "Hotels",
    "rooms": "Rooms",
    "bookings": "Bookings"
  }
}
```

Wie die Umschaltung funktioniert

```
Benutzer klickt [عربي] im LanguageSwitcher
    ↓
setLanguage('ar') wird aufgerufen
    ↓
LanguageProvider:
    ├── State wird auf 'ar' gesetzt
    ├── localStorage.setItem('language', 'ar')    ← Speichert für nächsten Besuch
    ├── document.documentElement.lang = 'ar'      ← <html lang="ar">
```

```
    ├── document.documentElement.dir = 'rtl'      ← <html dir="rtl">>
    └── Alle Komponenten re-rendern mit t() → Arabische Texte
```

RTL-Unterstützung

Wenn `dir="rtl"` auf dem HTML-Element gesetzt ist, wird die gesamte Textrichtung umgekehrt. Zusätzlich werden Tailwind-Klassen wie `rtl:flex-row-reverse` aktiv, die das Layout für Arabisch anpassen.

---

14. Styling und Design

Farbsystem

Definiert in: `tailwind.config.js`

```
Brand-Farbe:  #C17817 (Gold/Orange) - Hauptakzentfarbe
              ├── brand-50:  #FDF6EC  (sehr hell)
              ├── brand-100: #FAECD5
              ├── brand-200: #F3D4A3
              ├── brand-300: #EBB96E
              ├── brand-400: #D9952F
              ├── brand-500: #C17817  ← Standard (class="bg-brand")
              ├── brand-600: #A66413  ← Hover-Zustand
              ├── brand-700: #7E4C0F
              ├── brand-800: #56350B
              └── brand-900: #2F1D06  (sehr dunkel)

Charcoal:     #1c1c1c - Dunkler Hintergrund (Navigation, Footer)
```

Schriftarten

Definiert in: `src/app/layout.js`

```
Überschriften: Playfair Display (Serif, elegant)
               → class="font-heading"

Fließtext:     Inter (Sans-Serif, gut lesbar)
               → class="font-sans" (Standard für body)
```

Vordefinierte CSS-Klassen

Definiert in: `src/app/globals.css`

Klasse	Beschreibung
`.btn`	Basis-Button (Padding, Border-Radius, Cursor)
`.btn-primary`	Blauer Button (#007bff)
`.btn-success`	Grüner Button (#28a745)
`.btn-danger`	Roter Button (#dc3545)
`.btn-secondary`	Grauer Button (#6c757d)
`.form-group`	Formular-Feld mit Label + Input
`.card`	Karte mit Schatten und Hover-Effekt

`.table`	Formatierte Tabelle
`.container`	Zentrierter Container mit Padding
`.error`	Rote Fehlermeldung
`.success`	Grüne Erfolgsmeldung
`.back-link`	"← Zurück"-Link
`.section-label`	Kleine Überschrift (z.B. "EXPLORE YEMEN")
`.section-heading`	Große Abschnitts-Überschrift

Responsive Design

Die App ist **mobile-first** gestaltet. Tailwind-Breakpoints:

- `sm:` → ab 640px Breite
- `md:` → ab 768px Breite (Tablet)
- `lg:` → ab 1024px Breite (Desktop)

Beispiel:

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3">
```

- Handy: 1 Spalte
- Tablet: 2 Spalten
- Desktop: 3 Spalten

---

15. Middleware und Routenschutz

Was ist Middleware?

Middleware ist Code, der **vor** dem Laden jeder Seite ausgeführt wird. Sie sitzt "zwischen" dem Browser und der Seite.

Datei: `src/middleware.js`

```
import { updateSession } from '@/lib/supabase/middleware'

export async function middleware(request) {
  return await updateSession(request)
}

export const config = {
  matcher: [
    // Alles AUSSER statische Dateien (Bilder, CSS, JS)
    '/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)',
  ],
}
```

Datei: `src/lib/supabase/middleware.js` (updateSession)

```
Jede HTTP-Anfrage
    ↓
1. Supabase-Session aus Cookies erneuern
```

```
   → Wichtig: Sessions laufen ab und müssen regelmäßig aufgefrischt werden
    ↓
2. Aktuellen Benutzer ermitteln
   → supabase.auth.getUser()
    ↓
3. Ist der Pfad geschützt?
   → /my-bookings → JA
   → /admin       → JA
   → /hotel/...   → NEIN (öffentlich)
    ↓
4a. Geschützter Pfad + KEIN Benutzer:
    → Weiterleitung zu /login?redirect=/my-bookings
    (Der redirect-Parameter merkt sich, wohin der Benutzer wollte)
    ↓
4b. Admin-Pfad + Benutzer ist KEIN Admin:
    → ADMIN_EMAILS aus Umgebungsvariablen lesen
    → E-Mail des Benutzers prüfen
    → Nicht in der Liste → Weiterleitung zur Startseite
    ↓
5. Alles OK → Seite wird normal geladen
```

---

16. Bild-Upload

Wie der Upload funktioniert

API-Route: `src/app/api/upload/route.js`

```
Admin-Panel: Benutzer wählt ein Bild aus
    ↓
Browser sendet FormData:
    ├── file: Die Bilddatei
    └── bucket: "hotel-images" oder "room-images"
    ↓
Server (api/upload/route.js):
    ├── Auth-Prüfung: Nur eingeloggte Benutzer
    ├── Typ-Prüfung: Nur JPEG, PNG, WebP, GIF
    ├── Größen-Prüfung: Max 5MB
    ├── Dateiname generieren: "1706000000-x7k2m3.jpg"
    │   (Zeitstempel + Zufallsstring = einzigartig)
    ├── In Supabase Storage hochladen
    └── Öffentliche URL zurückgeben
    ↓
Browser: Speichert URL im Formular → Wird bei Hotel/Zimmer-Erstellung mitgespeichert
```

Supabase Storage Buckets

```
Supabase Storage
├── hotel-images/
│   ├── 1706000000-x7k2m3.jpg
│   ├── 1706000001-a8b9c2.png
│   └── ...
└── room-images/
    ├── 1706000002-d4e5f6.jpg
    └── ...
```

---

17. E-Mail-System

Buchungsbestätigungen

Datei: `src/lib/email.js`

Dienst: Resend (API-basierter E-Mail-Versand)

```
Neue Buchung wird erstellt (api/bookings/route.js)
    ↓
sendBookingConfirmation({
    to: "gast@email.de",
    guestName: "Max Mustermann",
    hotelName: "Grand Hotel Sanaa",
    roomName: "Deluxe Suite",
    pricePerNight: 120,
    checkIn: "2026-03-01",
    checkOut: "2026-03-05",
    totalPrice: 480
})
    ↓
resend.emails.send({
    from: 'Hotel Booking <onboarding@resend.dev>',
    to: 'gast@email.de',
    subject: 'Booking Confirmation - Grand Hotel Sanaa',
    html: '...'   ← HTML-Template mit allen Buchungsdetails
})
```

Wichtig:

- Der E-Mail-Versand ist **"Fire-and-Forget"**: Er blockiert NICHT die Buchungsantwort

  ```
  sendBookingConfirmation({...}).catch(err => console.error(err))
  // ↑ Buchung wird sofort bestätigt, E-Mail kommt im Hintergrund
  ```

- Wenn `RESEND_API_KEY` nicht gesetzt ist, wird KEINE E-Mail gesendet (nur Console-Log)
- Dies ist **optional** - die Buchung funktioniert auch ohne E-Mail

---

18. Datenfluss-Diagramme

Datenfluss: Startseite laden

```
Browser öffnet /
    │
    ↓
Middleware: Session prüfen (/ ist öffentlich → durchlassen)
    │
    ↓
Server Component: src/app/page.js
    │
    ├── createClient() → Supabase Server-Client
    │
    ├── supabase.from('hotels').select('*, rooms(price)')
    │   → Supabase Datenbank
    │   ← Alle Hotels mit Zimmerpreisen
```

```
    │
    ├── Mindestpreis pro Hotel berechnen
    ├── Städte mit Hotelanzahl zählen
    │
    ├── HTML rendern mit:
    │   ├── HeroSearch(cities)       ← Client Component
    │   ├── StatsRow(hotelCount)     ← Client Component
    │   ├── DestinationCards(destinations) ← Client Component
    │   ├── FeaturedHotels(hotels)   ← Client Component
    │   └── AboutSection()           ← Client Component
    │
    ↓
HTML wird an den Browser gesendet
    │
    ↓
Browser: React "hydriert" die Client Components
    │
    ├── LanguageProvider startet → Sprache aus localStorage laden
    ├── AuthProvider startet → Benutzer aus Session laden
    ├── Navigation rendert → nutzt useAuth() und useLanguage()
    └── Seite ist interaktiv
```

Datenfluss: Hotel buchen

```
RoomList: Benutzer klickt "Book Now"
    │
    ├── sessionStorage ← Hotel + Zimmer gespeichert
    ├── useAuth().user vorhanden?
    │   ├── JA → router.push('/booking')
    │   └── NEIN → router.push('/booking/checkout-choice')
    │
    ↓
/booking (Client Component):
    │
    ├── sessionStorage → Hotel + Zimmer lesen
    ├── useAuth().user → E-Mail vorausfüllen
    │
    ├── Benutzer füllt Formular aus
    ├── Klickt "Jetzt buchen"
    │
    ├── fetch('POST /api/bookings', { room_id, firstname, ... })
    │       │
    │       ↓
    │   Server: api/bookings/route.js
    │       │
    │       ├── Benutzer prüfen (optional)
    │       ├── Felder validieren
    │       ├── Verfügbarkeit prüfen (Supabase Query)
    │       ├── Buchung in DB speichern
    │       ├── E-Mail senden (Hintergrund)
    │       └── ← JSON zurück (201)
    │
    ├── sessionStorage ← Buchungsdaten speichern
    ├── sessionStorage: Hotel + Zimmer löschen
    │
    └── router.push('/booking/confirmation')
```

```
            │
            ↓
/booking/confirmation:
    │
    ├── sessionStorage → Buchungsdaten lesen + löschen
    └── Bestätigung anzeigen
```

---

19. Wie Komponenten miteinander verbunden sind

Hierarchie aller Komponenten

```
layout.js (Root Layout)
│
├── LanguageProvider ··********** Stellt t(), language, isRTL bereit
│   │
│   └── AuthProvider ··********** Stellt user, loading, signOut bereit
│       │
│       ├── Navigation ******** Nutzt: useAuth(), useLanguage()
│       │   │                   Zeigt: Logo, Links, Login/Logout, LanguageSwitcher
│       │   │
│       │   └── LanguageSwitcher · Nutzt: useLanguage()
│       │                         Zeigt: EN | عربي Buttons
│       │
│       ├── {children} ******** Die aktuelle Seite:
│       │   │
│       │   ├── page.js (Startseite) ****** Server Component
│       │   │   ├── HeroSearch ************ Client, nutzt: useLanguage()
│       │   │   ├── StatsRow ************** Client, nutzt: useLanguage()
│       │   │   ├── DestinationCards ****** Client, nutzt: useLanguage()
│       │   │   ├── FeaturedHotels ******** Client, nutzt: useLanguage()
│       │   │   └── AboutSection ********** Client, nutzt: useLanguage()
│       │   │
│       │   ├── hotel/[id]/page.js ******** Server Component
│       │   │   └── RoomList ************** Client, nutzt: useAuth(), useLanguage()
│       │   │                              Schreibt: sessionStorage
│       │   │
│       │   ├── booking/page.js ********** Client
│       │   │   │                          Nutzt: useAuth()
│       │   │   │                          Liest: sessionStorage
│       │   │   │                          Ruft: POST /api/bookings
│       │   │   │
│       │   │   ├── checkout-choice ******* Client
│       │   │   │                          Navigation zu login/register/booking
│       │   │   │
│       │   │   └── confirmation ********** Client
│       │   │                              Liest: sessionStorage (einmalig)
│       │   │
│       │   ├── login/page.js ************ Client
│       │   │                              Nutzt: Supabase Browser-Client
│       │   │                              Ruft: signInWithPassword()
│       │   │
│       │   ├── register/page.js ********** Client
│       │   │                              Nutzt: Supabase Browser-Client
│       │   │                              Ruft: signUp()
│       │   │
```

```
│       │   ├── my-bookings/page.js ····· Client (geschützt)
│       │   │                            Nutzt: useAuth()
│       │   │                            Ruft: GET/PUT/DELETE /api/bookings
│       │   │
│       │   └── admin/ ···................ (eigenes Layout, geschützt)
│       │       ├── admin/layout.js ····· Server Component
│       │       │   └── AdminSidebar ····· Client, nutzt: useLanguage()
│       │       │
│       │       ├── admin/page.js ····· Dashboard
│       │       │                        Ruft: GET /api/hotels, rooms, bookings
│       │       │
│       │       ├── admin/hotels/page.js ·· CRUD für Hotels
│       │       │                        Ruft: GET/POST/PUT/DELETE /api/hotels
│       │       │                        Ruft: POST /api/upload
│       │       │
│       │       ├── admin/rooms/page.js ··· CRUD für Zimmer
│       │       │                        Ruft: GET/POST/PUT/DELETE /api/rooms
│       │       │                        Ruft: POST /api/upload
│       │       │
│       │       └── admin/bookings/page.js  CRUD für Buchungen
│       │                                  Ruft: GET/PUT/DELETE /api/bookings
│       │
│       └── Footer ................... Nutzt: useLanguage()
│                                      Zeigt: Links, Kontakt, Social Media
│
```

Verbindungen zwischen Dateien

```
                        ┌──────────────────┐
                        │     layout.js    │
                        │ (importiert alles)│
                        └──────────┬───────┘
                                   │
              ┌────────────────────┼────────────────────┐
              │                    │                    │
              ▼                    ▼                    ▼
┌──────────────────┐ ┌──────────────┐ ┌────────────────┐
│ AuthProvider.js  │ │LanguageProvider│ │ globals.css    │
│                  │ │              │ │                │
│ importiert:      │ │ importiert:  │ │ Tailwind +     │
│ └─ client.js     │ │ └─ en.json   │ │ eigene Stile   │
│    (Supabase     │ │ └─ ar.json   │ └────────────────┘
│     Browser)     │ └──────────────┘
└──────────────────┘

┌──────────────────┐
│ middleware.js    │ Importiert:
│                  │ └─ middleware.js (supabase)
│ Wird AUTOMATISCH │     └─ Importiert: @supabase/ssr
│ von Next.js      │     └─ Importiert: next/server
│ aufgerufen       │
└──────────────────┘

┌──────────────────┐
│ API-Routen       │ Importieren alle:
│ /api/hotels      │ └─ server.js (Supabase Server-Client)
```

```
    │ /api/rooms        │ └─ next/server (NextResponse)
    │ /api/bookings     │
    │ /api/upload       │ /api/bookings importiert zusätzlich:
    └───────────────────┘   └─ email.js (Buchungsbestätigung)
```

---

20. Projekt starten

Voraussetzungen

- **Node.js** installiert (v18 oder höher)
- **npm** (kommt mit Node.js)

Befehle

```
# 1. Abhängigkeiten installieren
npm install

# 2. Entwicklungsserver starten
npm run dev
# → App läuft auf http://localhost:3000

# 3. Produktions-Build erstellen
npm run build

# 4. Produktions-Build starten
npm start

# 5. Code-Qualität prüfen
npm run lint
```

Umgebungsvariablen (.env.local)

Diese Datei enthält die geheimen Konfigurationswerte:

```
# Supabase-URL - Wohin die Datenbank-Anfragen gehen
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co

# Supabase öffentlicher Schlüssel - Erlaubt Lesezugriff
NEXT_PUBLIC_SUPABASE_ANON_KEY=xxx

# Admin-E-Mails - Kommagetrennte Liste von Admin-Benutzern
ADMIN_EMAILS=admin@example.com

# Resend API-Schlüssel - Für E-Mail-Versand (optional)
RESEND_API_KEY=xxx
```

NEXT_PUBLIC_ Prefix bedeutet: Diese Variable ist auch im Browser sichtbar (wird in den Client-Code eingebaut). Variablen OHNE dieses Prefix sind nur auf dem Server verfügbar.

Glossar

Begriff	Erklärung
Component	Wiederverwendbarer UI-Baustein in React

Hook	Spezielle React-Funktion (z.B. useState, useEffect)
State	Daten, die sich ändern können und die UI aktualisieren
Props	Daten, die von Eltern- an Kind-Komponente übergeben werden
Context	System zum Teilen von Daten über viele Komponenten hinweg
Provider	Komponente, die Context-Daten bereitstellt
Server Component	Wird auf dem Server gerendert (Standard in Next.js)
Client Component	Wird im Browser gerendert (`'use client'`)
API Route	Backend-Endpunkt in Next.js (`src/app/api/`)
Middleware	Code, der vor jeder Seitenladung ausgeführt wird
CRUD	Create, Read, Update, Delete - die 4 Grundoperationen
JWT	JSON Web Token - zur Authentifizierung
Session	Verbindung zwischen Browser und Server (Login-Zustand)
sessionStorage	Temporärer Speicher im Browser (überlebt kein Tab-Schließen)
localStorage	Dauerhafter Speicher im Browser (überlebt Tab-Schließen)
RTL	Right-to-Left - Textrichtung für Arabisch
LTR	Left-to-Right - Textrichtung für Deutsch/Englisch
SSR	Server-Side Rendering - Seite wird auf Server vorgerendert
Hydration	Prozess, bei dem React die Server-HTML im Browser interaktiv macht
Route	URL-Pfad in der App (z.B. /hotel/abc)
Dynamic Route	Route mit Variable (z.B. [id] in /hotel/[id])
Fetch	Browser-API zum Senden von HTTP-Anfragen
ENV / Environment Variable	Konfigurationswerte außerhalb des Codes
Bucket	Ordner/Container in Supabase Storage für Dateien
FK (Foreign Key)	Fremdschlüssel - Verbindung zwischen Datenbanktabellen
PK (Primary Key)	Primärschlüssel - Eindeutige ID eines Datensatzes